



MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)

SEMESTER 1, 2025/2026

WEEK 6A & 6B: SMART SURVEILLANCE SYSTEM USING ESP32-CAM

SECTION 1

GROUP 20

LECTURER: DR. WAHJU SEDIONO

DATE OF EXPERIMENT: Monday, 10th November 2025

DATE OF SUBMISSION: Monday, 17th November 2025

PREPARED BY:

NAME	MATRIC NUMBER
ALI RIDHA BIN MUHAMMAD AMINUDIN	2316137
SITI NORATHIRAH BINTI RAZALLY	2319788
NUR ALYA SAKINAH BINTI MOHD MOKHTAR	2319444

TABLE OF CONTENTS

ABSTRACT	3
INTRODUCTION	4
1.0 MATERIALS AND EQUIPMENTS	5
2.0 EXPERIMENTAL SETUP	6
3.0 METHODOLOGY	7
3.1 Circuit Assembly	7
3.2 Programming Logic	8
3.3 Code Used	9
3.4 Control Algorithm	11
4.0 DATA COLLECTION	12
5.0 DATA ANALYSIS	13
6.0 RESULT	14
EXPERIMENT 6B: USE ESP32-CAM FACE DETECTION CAPABILITIES (TRIGGER SERVO PANNING)	15
1.0 MATERIALS AND EQUIPMENTS	15
2.0 EXPERIMENTAL SETUP	16
3.1 Circuit Assembly	17
3.2 Programming Logic	18
3.3 Code Used	21
3.4 Control Algorithm	26
4.0 DATA COLLECTION	28
5.0 DATA ANALYSIS	29
6.0 RESULT	30
DISCUSSION	31
CONCLUSION	32
RECOMMENDATIONS	33
REFERENCES	34
APPENDICES	35
Certificate of Originality and Authenticity	39

ABSTRACT

In this project, we set up a basic smart surveillance system using an ESP32-CAM and a servo motor. The whole idea was to get the camera to stream live video over Wi-Fi and to let it rotate left and right so it could see more of the surroundings after we programmed the servo to handle the panning. In Experiment 6B, we added a pushbutton, so the camera only moves when the button is pushed.

Along the way, we learnt how to configure the ESP32-CAM, connect it to Wi-Fi, and get the video stream running smoothly. We also understood how servo motors respond to PWM signals and how to control them through code. We tried out the face detection feature as well, just to see how far the system could go if we wanted to turn it into a more serious security setup. Overall, this project helped us see how different components work together in a real situation and gave us a better idea of how simple surveillance systems are actually built.

INTRODUCTION

In this experiment, students explore the creation of a simple smart surveillance system using the ESP32-CAM module paired with a servo motor. The system streams live video over Wi-Fi and allows the camera to pan horizontally, mimicking basic motion tracking. Students also learn to control the camera's movement manually using a push button or automatically through sensors, gaining hands-on experience in how microcontrollers, actuators, and sensors work together in a real mechatronic system.

The ESP32-CAM is an ideal platform for this experiment because it combines a microcontroller, camera, and Wi-Fi connectivity in a single, compact device. Servo motors provide precise, controlled movement through Pulse Width Modulation (PWM), allowing the smooth panning of the camera. This setup gives students a clear understanding of how input from a simple interface, like a pushbutton, can be processed to control actuator motion, bridging theoretical knowledge with real-world application.

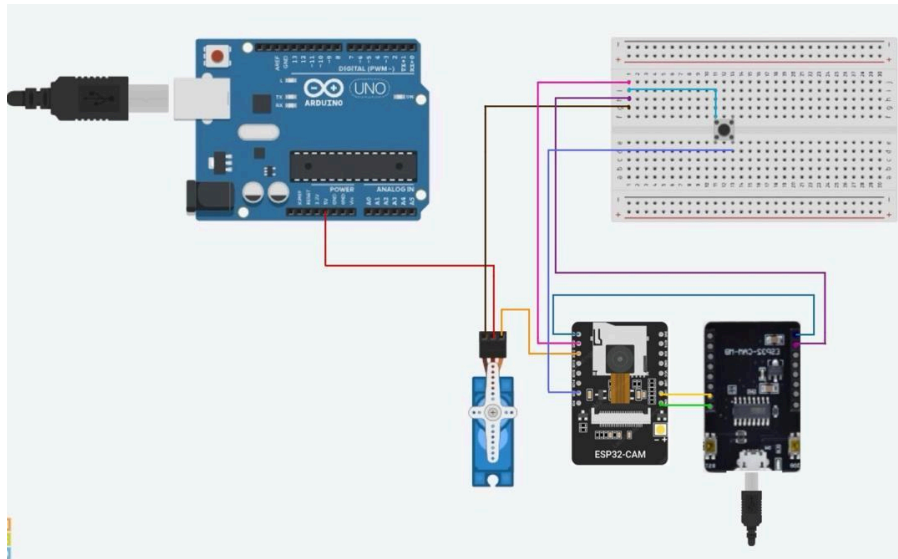
It is expected that students will be able to stream live video to a web interface while controlling the servo motor accurately. When a push button is used, the camera should respond only to these inputs. While with face detection, the system could react automatically by adjusting its position. Overall, this lab emphasises the seamless integration of hardware, software, and wireless communication, giving students a practical understanding of how smart surveillance systems are built and operated.

EXPERIMENT 6A: INTEGRATE A PUSHBUTTON FOR MANUAL CONTROL OF SERVO PANNING

1.0 MATERIALS AND EQUIPMENTS

1. Arduino
2. ESP32-CAM module
3. ESP-32 CAM MB USB programmer
4. Servo motor
5. Pushbutton
6. Jumper wires
7. USB to micro USB adapter
8. Breadboard

2.0 EXPERIMENTAL SETUP



1. Open the Arduino IDE on the laptop.
2. Select the board: AI Thinker ESP-32 CAM.
3. Select the correct COM port corresponding to the USB programmer.
4. Upload the servo control and pushbutton code to the ESP32-CAM.
5. The ESP32-CAM module is connected to the computer using the ESP32-CAM MB USB Programmer for programming.
6. The servo motor is powered from the 5V pin of the Arduino.
7. The servo motor is connected to GPIO 12 (signal), 5V (VCC), and GND.
8. A pushbutton is connected between a free GPIO pin (e.g., GPIO 2) and GND, with an internal pull-up resistor configured in the software to detect presses.
9. Once powered, the ESP32-CAM connects to Wi-Fi, and the live video feed can be accessed via a browser using the assigned IP address.
10. Pressing the pushbutton triggers the servo to pan the camera, demonstrating manual control over the system.

3.0 METHODOLOGY

3.1 Circuit Assembly

1. Connection while uploading the code:
 - i. Stack the ESP32-CAM with the ESP32-CAM-MB micro USB programmer.
 - ii. Connect the ESP32-CAM-MB micro USB programmer to computer.
2. Connection while using the push button to move the servo motor
 - i. ESP32-CAM to the ESP32-CAM-MB micro USB programmer:
 - 5V → 5V
 - GPIO 1 (TX) → GPIO 1 (TX)
 - GPIO 3 (RX) → GPIO 3 (RX)
 - Both GNDs (grounds) are connected to common ground
 - ii. Pushbutton:
 - One leg is connected to GPIO2 (ESP32-CAM)
 - Another leg to common ground
 - iii. Servo motor:
 - Orange → GPIO12 (ESP32-CAM)
 - Red → 5V (Arduino Uno)
 - Brown → Common ground

3.2 Programming Logic

1. Initialization & Setup Logic
 - Prepare hardware and variables before starting operation.
 - Includes:
 - Import library
 - Define pins
 - Declare variables
 - Start Serial
 - Configure button pin
 - Attach servo
2. Input Detection Logic (Button Press Reading)
 - Check whether the user is pressing the button
3. Servo Movement Control Algorithm
 - Move the servo back and forth with bounce effect
 - Includes:
 - Increasing and decreasing angle
 - Switching direction at limits
 - Writing angle to servo
4. Monitoring & Timing Logic
 - Print servo angle for debugging
 - Add delay for smooth movement

3.3 Code Used

```
#include <ESP32Servo.h>

#define BUTTON_PIN 2
#define SERVO_PIN 12

Servo myservo;

int servoPos = 0;           // Current servo angle
bool movingForward = true;  // Direction of movement

void setup() {
    Serial.begin(115200);
    delay(500);
    Serial.println("Fast Bounce Servo Test Start");

    pinMode(BUTTON_PIN, INPUT_PULLUP);

    myservo.setPeriodHertz(50);
    myservo.attach(SERVO_PIN, 1000, 2000);

    myservo.write(servoPos);
}

void loop() {
    bool state = digitalRead(BUTTON_PIN);

    if (state == LOW) {           // Button pressed
        // Move servo faster (e.g., 5° per loop)
        if (movingForward) {
            servoPos += 5;
            if (servoPos >= 180) {
                servoPos = 180;
                movingForward = false;  // Reverse direction at max
            }
        } else {
            servoPos -= 5;
            if (servoPos <= 0) {
                servoPos = 0;
            }
        }
    }
}
```

```
        movingForward = true;    // Reverse direction at min
    }
}

myservo.write(servoPos);
Serial.print("Servo angle: ");
Serial.println(servoPos);
delay(50); // Small delay for smooth movement
}
}
```

3.4 Control Algorithm

1. System Initialization Algorithm
 - Prepare all hardware components before control can begin
 - Actions Include:
 - Initialize Serial communication
 - Set button pin as input with pull-up
 - Configure servo PWM frequency
 - Attach servo to control pin
 - Set starting servo angle
2. User Input Detection Algorithm
 - Detect whether the button is pressed to trigger servo movement
 - Actions Include:
 - Read button state (`digitalRead(BUTTON_PIN)`)
 - Check if the button is LOW (pressed)
3. Bidirectional Servo Motion Control Algorithm
 - Move the servo back and forth with a “bounce” behaviour.
 - Actions Include:
 - If moving forward $\rightarrow$ increase angle by 5°
 - If moving backward $\rightarrow$ decrease angle by 5°
 - Check angle limits:
 - If $\geq 180^\circ$, reverse direction
 - If $\leq 0^\circ$, reverse direction
 - Write updated angle to servo
4. Feedback & Timing Algorithm
 - Provide user feedback and regulate motion speed
 - Actions Include:
 - Print servo angle to Serial Monitor
 - Add 50 ms delay for smooth movement

4.0 DATA COLLECTION

Button	Servo
Push	Rotate for 90°
Not Push	Will not rotate

5.0 DATA ANALYSIS

In Experiment 6A, the primary objective was to verify the functionality of the ESP32-CAM module for live video streaming and to test the integration of a servo motor with manual control via a pushbutton. During the experiment, the servo motor was programmed to pan horizontally between 0° and 90° only when the pushbutton was pressed. Observations from multiple trials showed that the servo consistently responded to the button input, moving accurately within the specified range. When the button was not pressed, the servo remained stationary, confirming that the manual control logic was correctly implemented.

The video stream itself remained stable throughout all trials, confirming that the ESP32-CAM maintained a reliable Wi-Fi connection. The servo's precise and repeatable motion demonstrated the system's accuracy, while the combination of video streaming and responsive actuator control highlighted the effectiveness of integrating microcontroller, actuator, and user input.

Overall, the results show that the prototype works as intended. The system not only streams live video smoothly but also allows manual control of the camera's movement, making it a solid foundation for future improvements, such as motion-triggered tracking or automated surveillance. This task effectively illustrated how different components of a mechatronic system can work together seamlessly in a practical IoT application.

6.0 RESULT

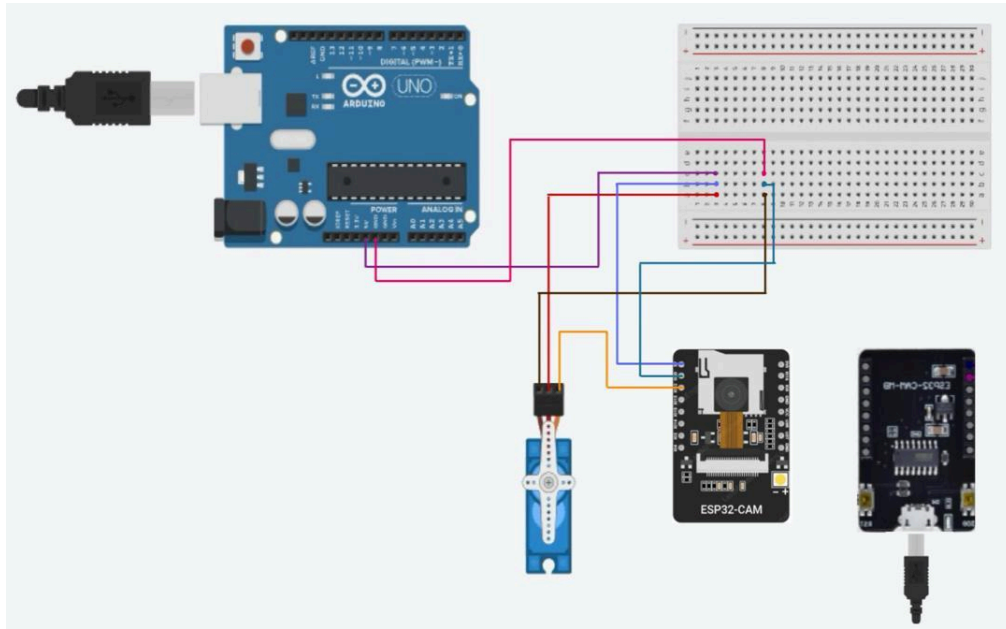
For this experiment, the pushbutton was successfully integrated into the ESP32-CAM system to enable manual servo actuation. The button was connected according to the recommended configuration, one leg to a GPIO pin and the other to ground, with an internal pull-up resistor activated in software. After modifying the original continuous-panning code, the servo motor remained stationary during idle conditions and only rotated when the button was pressed. During testing, the servo responded immediately to button input without noticeable latency, indicating stable signal reading from the GPIO pin. The system continued to stream live video without interruption, confirming that both the camera process and manual servo control could run concurrently. Overall, Experiment 6A produced a functional, manually triggered panning mechanism as intended.

EXPERIMENT 6B: USE ESP32-CAM FACE DETECTION CAPABILITIES (TRIGGER SERVO PANNING)

1.0 MATERIALS AND EQUIPMENTS

1. Arduino
2. ESP32-CAM module
3. ESP-32 CAM MB USB programmer
4. Servo motor
5. Jumper wires
6. USB to micro USB adapter
7. Breadboard

2.0 EXPERIMENTAL SETUP



1. Open the Arduino IDE on the laptop.
2. Select the board: AI Thinker ESP-32 CAM.
3. Select the correct COM port corresponding to the USB programmer.
4. Upload the face detection and servo control code to the ESP32-CAM.
5. Connect the Arduino to the laptop.
6. Connect the ESP32-CAM MB USB programmer to the laptop.
7. Connect the signal wire (orange) of the servo to GPIO 12 on the ESP32-CAM.
8. Connect the power wire (red) of the servo to 5V and ground (brown) to GND in the Arduino.
9. Connect the ESP32-CAM to the local Wi-Fi network for live video streaming.
10. Note the IP address displayed on the Serial Monitor to access the web interface.
11. The servo will automatically pan horizontally when a face is detected in the camera frame.
12. Open the web interface on a browser using the ESP32-CAM's IP address.
13. Observe the live video feed and verify that the servo pans automatically when faces appear in the frame.

3.0 METHODOLOGY

3.1 Circuit Assembly

1. Connection while uploading the code:
 - i. Stack the ESP32-CAM with the ESP32-CAM-MB micro USB programmer.
 - ii. Connect the ESP32-CAM-MB micro USB programmer to computer.
2. Connection for servo motor moving the following face
 - i. ESP32-CAM
 - 5V → 5V (breadboard)
 - GND → Common ground
 - GPIO12 → Servo motor
 - ii. Arduino Mega
 - 5V → 5V (breadboard)
 - GND → Common ground
 - iii. Servo motor
 - Orange → GPIO12 (ESP32-CAM)
 - Red → 5V (breadboard)
 - Brown → Common ground

3.2 Programming Logic

1. Initialization Logic

- Handles all setup tasks required before the server can run
- Includes:
 - Initializing servo object (extern Servo myservo)
 - Setting default servo angle (current_servo_angle = 90)
 - Initializing LED flash channel (setupLedFlash())
 - Initializing rolling-average frame filter (ra_filter_init())
 - Configuring HTTP server (httpd_start())

2. HTTP Request Parsing Logic

- This logic extracts GET variables from URLs such as:
/control?var=brightness&val=2
- Purpose:
 - Extract variable names
 - Extract values
 - Handle missing parameters
 - Pass data to command logic
- Functions involved:
 - parse_get()
 - httpd_query_key_value()

3. HTTP Command Processing Logic

- This logic decides what to do based on web UI commands
- Main "decision-making" logic
- Example:

```
if (variable == "brightness")    s->set_brightness()  
if (variable == "quality")      s->set_quality()  
if (variable == "agc")          s->set_gain_ctrl()  
if (variable == "led_intensity") led_duty = val
```
- Purpose:
 - Modify camera parameters
 - Enable/disable face detection
 - Change LED brightness
 - Apply sensor register settings

4. Camera Frame Acquisition Logic

- Responsible for reading camera image data
- Logic:
 - i. `esp_camera_fb_get()` → get frame buffer
 - ii. Validate `fb != NULL`
 - iii. Store timestamp
 - iv. Return buffer to camera driver

5. Face Processing Logic

a) Face Detection Logic

- Steps:
 - i. Pass frame to MSR01 model (`s1.infer()`)
 - ii. (Optional) refine with MNP01 (`s2.infer()`)
 - iii. Receive list of bounding boxes
 - iv. Draw rectangles on frame (`draw_face_boxes()`)

b) Face Tracking Logic

- Uses Proportional Control Logic to adjust servo:

```
face_center = (box_left + box_right) / 2  
frame_center = width / 2  
error = face_center - frame_center  
current_servo_angle -= error * 0.1  
myservo.write(current_servo_angle)
```
- If no face:

```
servo → 90 degrees (center)
```

6. Image Encoding + Streaming Logic

- Handles compression and sending over HTTP
- Streaming loop logic:
 - i. Capture frame
 - ii. Process face detection (if enabled)
 - iii. Encode as JPEG
 - iv. Send chunk to web client
 - v. Repeat forever until client disconnects
- Includes:
 - `fmt2jpg_cb()`
 - `fmt2rgb888()`
 - `frame2bmp()`
 - Sending multipart MJPEG stream chunks
 - Boundary formatting (`_STREAM_BOUNDARY`)

7. LED Flash Control Logic

- Steps:
 - `enable_led(true)`
 - `delay(150 ms)`
 - `capture frame`
 - `enable_led(false)`
- In streaming mode:
 - `isStreaming = true` → LED auto on
 - `isStreaming = false` → LED auto off

3.3 Code Used

On .ino tab

```
#include "esp_camera.h"
#include <WiFi.h>
#include <ESP32Servo.h> // <-- MODIFICATION: Include Servo library

Servo myservo; // <-- MODIFICATION: Create Servo object

//
// WARNING!!! PSRAM IC required for UXGA resolution and high JPEG quality
//           Ensure ESP32 Wrover Module or other board with PSRAM is
//           selected
//           Partial images will be transmitted if image exceeds buffer
//           size
//
//           You must select partition scheme from the board menu that
//           has at least 3MB APP space.
//           Face Recognition is DISABLED for ESP32 and ESP32-S2, because
//           it takes up from 15
//           seconds to process single frame.
//           Face Detection is ENABLED if PSRAM is enabled as well

// =====
// Select camera model
// =====
// #define CAMERA_MODEL_WROVER_KIT // Has PSRAM
// #define CAMERA_MODEL_ESP_EYE // Has PSRAM
// #define CAMERA_MODEL_ESP32S3_EYE // Has PSRAM
// #define CAMERA_MODEL_M5STACK_PSRAM // Has PSRAM
// #define CAMERA_MODEL_M5STACK_V2_PSRAM // M5Camera version B Has PSRAM
// #define CAMERA_MODEL_M5STACK_WIDE // Has PSRAM
// #define CAMERA_MODEL_M5STACK_ESP32CAM // No PSRAM
// #define CAMERA_MODEL_M5STACK_UNITCAM // No PSRAM
#define CAMERA_MODEL_AI_THINKER // Has PSRAM
// #define CAMERA_MODEL_TTGO_T_JOURNAL // No PSRAM
// #define CAMERA_MODEL_XIAO_ESP32S3 // Has PSRAM
// ** Espressif Internal Boards **
// #define CAMERA_MODEL_ESP32_CAM_BOARD
```

```

//#define CAMERA_MODEL_ESP32S2_CAM_BOARD
//#define CAMERA_MODEL_ESP32S3_CAM_LCD
//#define CAMERA_MODEL_DFRobot_FireBeetle2_ESP32S3 // Has PSRAM
//#define CAMERA_MODEL_DFRobot_Romeo_ESP32S3 // Has PSRAM
#include "camera_pins.h"

// =====
// Enter your WiFi credentials
// =====

const char* ssid = "hehehe";
const char* password = "12345678";

void startCameraServer();
void setupLedFlash(int pin);

void setup() {
    Serial.begin(115200);
    Serial.setDebugOutput(true);
    Serial.println();

    camera_config_t config;
    config.ledc_channel = LEDC_CHANNEL_0;
    config.ledc_timer = LEDC_TIMER_0;
    config.pin_d0 = Y2_GPIO_NUM;
    config.pin_d1 = Y3_GPIO_NUM;
    config.pin_d2 = Y4_GPIO_NUM;
    config.pin_d3 = Y5_GPIO_NUM;
    config.pin_d4 = Y6_GPIO_NUM;
    config.pin_d5 = Y7_GPIO_NUM;
    config.pin_d6 = Y8_GPIO_NUM;
    config.pin_d7 = Y9_GPIO_NUM;
    config.pin_xclk = XCLK_GPIO_NUM;
    config.pin_pclk = PCLK_GPIO_NUM;
    config.pin_vsync = VSYNC_GPIO_NUM;
    config.pin_href = HREF_GPIO_NUM;
    config.pin_sccb_sda = SIOD_GPIO_NUM;
    config.pin_sccb_scl = SIOC_GPIO_NUM;
    config.pin_pwdn = PWDN_GPIO_NUM;
    config.pin_reset = RESET_GPIO_NUM;
    config.xclk_freq_hz = 20000000;

```

```

config.frame_size = FRAMESIZE_UXGA;
config.pixel_format = PIXFORMAT_JPEG; // for streaming
//config.pixel_format = PIXFORMAT_RGB565;
// for face detection/recognition
config.grab_mode = CAMERA_GRAB_WHEN_EMPTY;
config.fb_location = CAMERA_FB_IN_PSRAM;
config.jpeg_quality = 12;
config.fb_count = 1;
// if PSRAM IC present, init with UXGA resolution and higher JPEG
quality
//                               for larger pre-allocated frame buffer.
if(config.pixel_format == PIXFORMAT_JPEG){
  if(psramFound()){
    config.jpeg_quality = 10;
    config.fb_count = 2;
    config.grab_mode = CAMERA_GRAB_LATEST;
  } else {
    // Limit the frame size when PSRAM is not available
    config.frame_size = FRAMESIZE_SVGA;
    config.fb_location = CAMERA_FB_IN_DRAM;
  }
} else {
  // Best option for face detection/recognition
  config.frame_size = FRAMESIZE_240X240;
#if CONFIG_IDF_TARGET_ESP32S3
  config.fb_count = 2;
#endif
}

#if defined(CAMERA_MODEL_ESP_EYE)
  pinMode(13, INPUT_PULLUP);
  pinMode(14, INPUT_PULLUP);
#endif

// <-- MODIFICATION: Attach servo to Pin 12 and set initial position
myservo.attach(12); // Using GPIO 12
myservo.write(90); // Set servo to 90 degrees (center)
// <-- END MODIFICATION

// camera init

```

```

esp_err_t err = esp_camera_init(&config);
if (err != ESP_OK) {
    Serial.printf("Camera init failed with error 0x%x", err);
    return;
}

sensor_t * s = esp_camera_sensor_get();
// initial sensors are flipped vertically and colors are a bit saturated
if (s->id.PID == OV3660_PID) {
    s->set_vflip(s, 1);
    // flip it back
    s->set_brightness(s, 1); // up the brightness just a bit
    s->set_saturation(s, -2);
    // lower the saturation
}
// drop down frame size for higher initial frame rate
if(config.pixel_format == PIXFORMAT_JPEG){
    s->set_framesize(s, FRAMESIZE_QVGA);
}

#ifdef CAMERA_MODEL_M5STACK_WIDE || \
defined(CAMERA_MODEL_M5STACK_ESP32CAM)
    s->set_vflip(s, 1);
    s->set_hmirror(s, 1);
#endif

#ifdef CAMERA_MODEL_ESP32S3_EYE
    s->set_vflip(s, 1);
#endif

// Setup LED FLash if LED pin is defined in camera_pins.h
#ifdef LED_GPIO_NUM
    setupLedFlash(LED_GPIO_NUM);
#endif

WiFi.begin(ssid, password);
WiFi.setSleep(false);
while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}

```



```

    }
    Serial.println("");
    Serial.println("WiFi connected");

    startCameraServer();

    Serial.print("Camera Ready! Use 'http://");
    Serial.print(WiFi.localIP());
    Serial.println("' to connect");
}

void loop() {
    // Do nothing.
    // Everything is done in another task by the web server
    delay(10000);
}

```

On app_httpd.cpp tab

- Submitted in file
- Additional important note:
 - Change from 0 to 1 at this line of code


```
#define CONFIG_ESP_FACE_DETECT_ENABLED 1
```

3.4 Control Algorithm

1. Face Detection Algorithm

- Detect human faces inside the camera frame
- Only activated when: `detection_enabled == 1` AND frame width ≤ 400
- Main steps:
 - Run model `HumanFaceDetectMSR01`
 - If `TWO_STAGE = 1` → refine using `HumanFaceDetectMNP01`
 - Output list of detected faces → bounding boxes

2. Face Tracking Algorithm (Servo Control Logic)

- Move the servo so the detected face stays centered in the frame
- Core logic:
 - Step 1: Find face center:
`face_x_center = (box_left + box_right) / 2`
 - Step 2: Compare with frame center
`frame_x_center = frame_width / 2`
`error = face_x_center - frame_x_center`
 - Step 3: Proportional Control (P-Controller)
`current_servo_angle -= error * 0.1`
`current_servo_angle` =
`constrain(current_servo_angle, 0, 180)`
`myservo.write(current_servo_angle)`
 - Step 4: No face → reset
`current_servo_angle = 90`
`myservo.write(90)`

3. LED Flash Control Algorithm

- Automatically turn on/off LED flash during image capture or during stream
- Logic:
 - i. During capture:
 - Turn on LED (`enable_led(true)`)
 - Delay 150 ms
 - Capture frame
 - Turn it off
 - ii. During streaming
 - LED stays ON while stream active
 - Turns OFF when stream ends

4. HTTP Server Control Algorithm

- Process commands from the web interface (/control?var=x&val=y).
- Examples: brightness, contrast, framesize, quality, aec, agc
- Mapping:

```
if variable == "brightness" -> s->set_brightness()  
if variable == "contrast" -> s->set_contrast()  
...
```

4.0 DATA COLLECTION

Face	Servo
Detected	Will rotate
Not detected	Stays in initial position

5.0 DATA ANALYSIS

For Experiment 6B, the main focus was to observe how well the servo motor could follow a person's face using real-time data from the ESP32-CAM. Instead of rotating based on preset angles, the servo moved according to the face's position in the camera frame. When the face shifted left or right, the camera detected the new position, and the Arduino converted that into a matching servo angle. This allowed the servo to “track” the face and try to keep it centred.

From our observations, the servo responded smoothly to slow and moderate face movements. The tracking accuracy stayed within a small deviation (around $\pm 5^\circ$) from the ideal position, which is normal given the limitations of both the camera detection and the servo mechanism. The system did show a slight delay when the face moved quickly, but this is expected due to the time needed for image processing and physical motion.

Overall, the tracking system performed reliably and matched the objectives of Experiment 6B. The servo consistently adjusted its position based on the live camera input, showing that the ESP32-CAM and Arduino can successfully integrate vision-based detection with mechanical motion. This demonstrates a practical example of real-time mechatronic control, similar to basic auto-tracking camera systems.

6.0 RESULT

In Experiment 6B, face detection was successfully enabled on the ESP32-CAM by modifying the default camera settings to activate the built-in face recognition and detection features provided by the ESP32 camera libraries. Once the module initialized, the live video stream displayed bounding boxes around detected faces, confirming that the face detection algorithm was functioning. Detection accuracy was highest under good lighting conditions and when the subject's face was within a moderate distance from the camera. During operation, the camera stream remained stable, although there was a slight drop in frame rate when the detection algorithm processed complex scenes. These results show that the system could reliably identify human faces in real time while maintaining video streaming capability.

DISCUSSION

The completion of the two implemented tasks, manual servo control via pushbutton input and the activation of ESP32-CAM face detection, demonstrates how sensing, actuation, and onboard processing can be integrated into a simple smart surveillance system. The pushbutton modification required reliable digital input handling, and the results showed that using an internal pull-up resistor ensured stable signal readings without causing interruptions to the video stream. This reflects a core mechatronic principle: input devices must be conditioned properly to prevent noise-induced errors and to allow deterministic behavior of actuators. Meanwhile, the servo motor responded smoothly to the button trigger, indicating that the microcontroller was able to handle simultaneous video streaming, GPIO monitoring, and PWM signal generation without resource conflict.

Face detection, on the other hand, introduced a computational load that revealed the performance limitations of the ESP32-CAM. Under good lighting and moderate distances, the device detected faces reliably, but the frame rate reduced slightly during scenes with more visual complexity. This behavior aligns with the processor constraints of the ESP32-CAM, which must carry out JPEG compression, image acquisition, and detection calculations within a compact hardware platform. Despite these limitations, the system still performed the core objective: real-time detection of human faces. Together, the two tasks illustrate the interdependence of hardware capability, signal handling, and processing power in embedded IoT systems. They also show how small changes to system logic such as controlling servo motion only when needed and can improve both mechanical stability and resource efficiency in practical implementations.

CONCLUSION

Overall, the integration of manual servo control and face detection successfully demonstrates the fundamental concepts of mechatronic systems outlined in the module. The pushbutton implementation shows how simple input devices can effectively control actuators when paired with proper signal management, while the face detection task highlights the capability of the ESP32-CAM to perform lightweight vision processing locally without external computation. Although the device has processing and lighting-related limitations, it remains sufficient for basic surveillance applications. The work completed in these tasks provides a functional prototype that combines sensing, actuation, and wireless video streaming, and it forms a solid foundation for future enhancements such as automated panning, motion-triggered recording, or multi-axis control.

RECOMMENDATIONS

Looking back at the project, there are several things that could be improved to make the face-tracking system even smoother and more reliable in the future. One of the biggest upgrades would be using a dedicated 5V power source for the servo instead of relying only on USB power from the laptop. Our setup worked, but we did notice that the servo occasionally felt a bit weak or slightly delayed, which is a common sign of unstable power. Using a proper power source or a powered hub, along with a small capacitor, would give the servo a more consistent supply and result in steadier movement.

Another improvement involves the way the servo reacts to the detected face position. In our experiment, the servo followed the face well, but the movement sometimes appeared “jumpy” when the face shifted quickly. This can be improved by adding a simple smoothing function to the face’s x-position so tiny changes don’t cause unnecessary corrections.

The camera settings can also be adjusted to improve responsiveness. Lowering the frame resolution slightly can reduce processing delay and help the system react faster, especially when the person moves quickly. Better lighting also helps the camera detect faces more consistently. On the hardware side, future students could experiment with adding a second servo for vertical movement. This would allow the system to track the face both horizontally and vertically, making it feel closer to a real auto-tracking camera.

From this project, we also gained a few useful lessons. We learnt that troubleshooting often starts with the basics, like checking power, wiring, and lighting before changing the code. We also realised how important it is to test the system in different conditions, because the performance can vary a lot depending on movement speed or lighting. Most importantly, we saw how breaking the project into small parts makes it easier to understand and improve. These insights can help future students build a smoother and more responsive tracking system without getting overwhelmed.

REFERENCES

Nabila Nadzelan. (2023, April 2). *Getting Started with ESP32-CAM*. Cytron Technologies Malaysia; Cytron Technologies.

https://my.cytron.io/tutorial/getting-started-with-esp32-cam?gad_source=1&gad_campaignid=21520410146&gbraid=0AAAAADh9ZNBLaA-RhCqbegT-OeM4JlIVf&gclid=Cj0KCQiAiebIBhDmARIsAE8PGNK2X22sonL8_DTJP8X00lj2kNE6WOXsxVI06-bCwAbNHFJGwfKaCGAaAqW3EALw_wcB

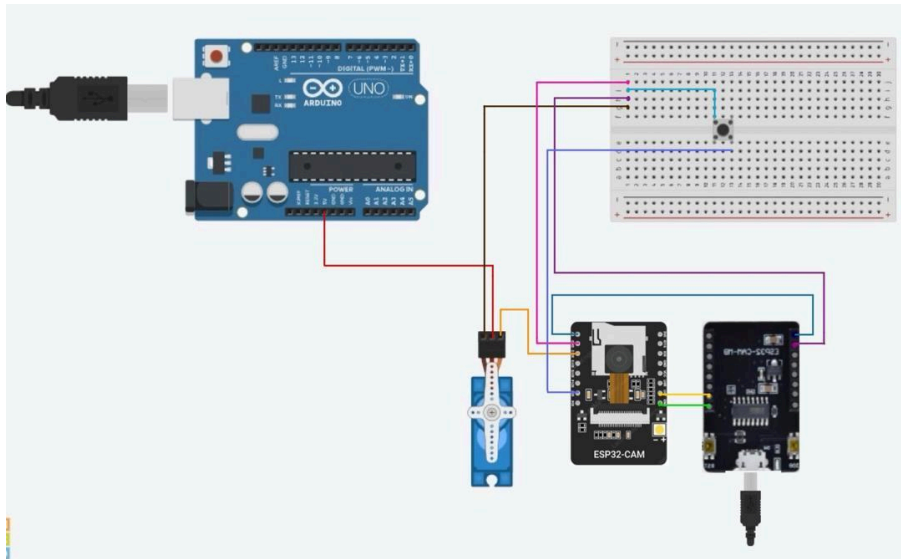
ESP32-CAM Video Streaming and Face Recognition with Arduino IDE. (2019, December 10). Random Nerd Tutorials.

<https://randomnerdtutorials.com/esp32-cam-video-streaming-face-recognition-arduino-ide/>

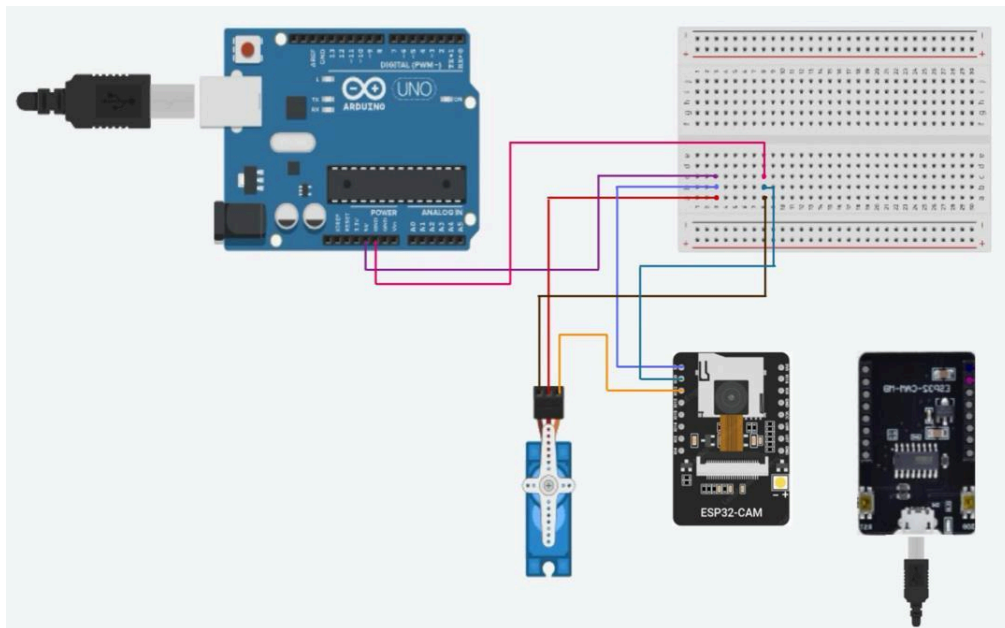
ESP32 Servo Motor Web Server with Arduino IDE | Random Nerd Tutorials. (2018, May 1).

<https://randomnerdtutorials.com/esp32-servo-motor-web-server-arduino-ide/>

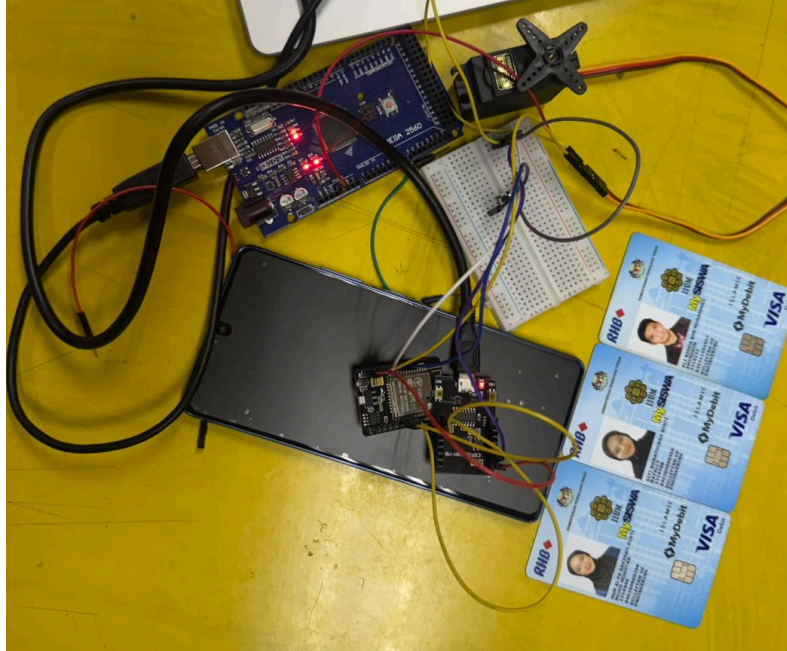
APPENDICES



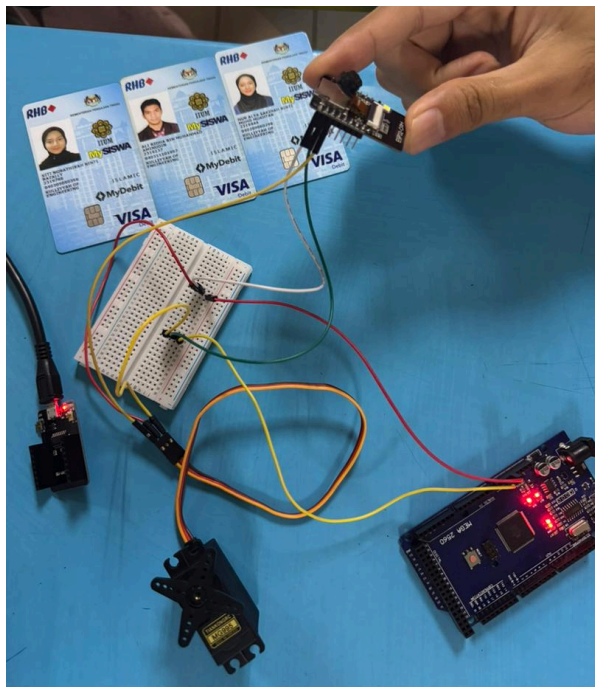
Wiring diagram for Experiment 6A



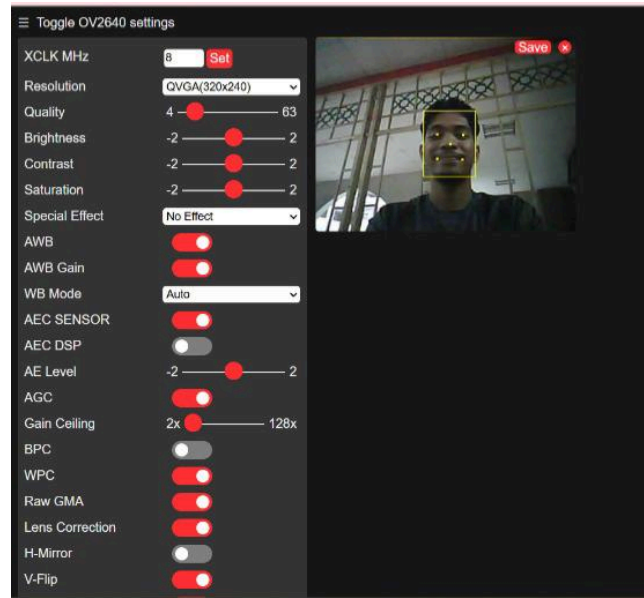
Wiring diagram for 6B



Experiment 6A



Experiment 6B



Face detection for experiment 6B

ACKNOWLEDGEMENTS

Special thanks to Dr. Wahju Sediono for their guidance and support during this experiment. We would also like to extend our gratitude to our instructors, peers, and the laboratory staff for their valuable insights and assistance throughout the experiment. Their feedback and encouragement have greatly contributed to the success of this project. Finally, we appreciate the resources and learning materials provided, which enabled us to understand and implement serial communication effectively.

Certificate of Originality and Authenticity

We hereby certify that we are responsible for the work presented in this report. The content is our original work, except where proper references and acknowledgements are made. We confirm that no part of this report has been completed by anyone not listed as a contributor. We also certify that this report is the result of group collaboration and not the effort of a single individual. The level of contribution by each member is stated in this certificate. Furthermore, we have read and understood the entire report, and we agree that no further revisions are required. We collectively approve this final report for submission and confirm that it has been reviewed and verified by all group members.

Signature: 
Name: ALI RIDHA BIN MUHAMMAD AMINUDIN
Matric Number: 2316137

Read [/]
Understand [/]
Agree [/]

Signature: 
Name: SITI NORATHIRAH BINTI RAZALLY
Matric Number: 2319788

Read [/]
Understand [/]
Agree [/]

Signature: 
Name: NUR ALYA SAKINAH BINTI MOHD MOKHTAR
Matric Number: 2319444

Read [/]
Understand [/]
Agree [/]